



Assignment No. 1

Title:

Hadoop Setup and Simple MapReduce

Aim:

To install and configure Apache Hadoop on a local machine or small cluster and to develop and execute a simple MapReduce program for analyzing sample data, thereby understanding the working of distributed storage and parallel data processing.

Objective:

The main objective of this lab is to understand the installation and configuration of Apache Hadoop on a local machine or small cluster environment. This experiment helps students learn how to set up and configure the Hadoop Distributed File System (HDFS), manage environment variables, and start Hadoop services successfully. Furthermore, students will develop and execute a simple MapReduce program to process and analyze sample data. Through this process, they will gain practical knowledge of the Map and Reduce phases, understand how distributed data processing works, and build a strong foundation for working with big data technologies and large-scale data analysis systems.

Software Requirements:

1. Operating System: Windows / Linux / macOS
2. Java Development Kit (JDK 8 or above)
3. Apache Hadoop (Version 3.x recommended)
4. SSH (for pseudo-distributed mode setup)
5. Hadoop Environment Variables Configuration
6. Command Line / Terminal
7. Text Editor or IDE (VS Code / Eclipse / IntelliJ IDEA)
8. Sample Dataset (Text file for MapReduce processing)

Outcome:

After completing this lab, students will be able to:

1. Install and configure Apache Hadoop.
2. Understand HDFS and the MapReduce framework.
3. Execute a simple MapReduce program on sample data.

Theory:

In modern computing, organizations generate extremely large volumes of data from sources such as social media, sensors, web applications, mobile devices, and transaction systems. This massive amount of information is known as **Big Data**. Big data is commonly characterized by the **three V's: Volume, Velocity, and Variety**. Volume refers to the huge amount of data generated, Velocity refers to the speed at which data is produced and processed, and Variety refers to different types of data such as structured, semi-structured, and unstructured data.

Traditional data processing systems are not capable of efficiently handling such large-scale datasets because they are limited in terms of storage capacity, processing power, and scalability. To address these limitations, distributed computing frameworks such as **Apache Hadoop** were developed. Hadoop is an open-source framework designed to store and process large datasets across clusters of computers. It provides features such as **scalability, fault tolerance, reliability, and distributed processing**.

Hadoop works on the concept of **distributed storage and parallel processing**, where large datasets are divided into smaller blocks and distributed across multiple machines (nodes) in a cluster. This architecture allows multiple nodes to process data simultaneously, significantly improving performance and efficiency. Hadoop is designed to run on **commodity hardware**, which means it can operate on low-cost machines rather than expensive high-performance systems.

One of the key advantages of Hadoop is its **fault tolerance**. Data in Hadoop is replicated across multiple nodes in the cluster. If one node fails, the system automatically retrieves the replicated data from another node, ensuring continuous operation without data loss.

Hadoop mainly consists of two fundamental components:

1.HDFS (Hadoop Distributed File System):

HDFS is responsible for storing large datasets in a distributed environment. It splits large files into smaller blocks (typically 128 MB or 256 MB) and distributes them across multiple nodes in the cluster. Each block is replicated on several nodes to ensure reliability and fault tolerance. HDFS follows a **master-slave architecture**, where the **NameNode** manages the file system metadata and tracks the location of data blocks, while **DataNodes** store the actual data blocks.

2. MapReduce:

MapReduce is the programming model used for processing large datasets in parallel. It divides a large task into smaller subtasks that run simultaneously across different nodes in the cluster. This parallel processing significantly improves efficiency when dealing with massive datasets. The



MapReduce framework works in several stages:

Map Phase:

In this phase, the input data is divided into smaller chunks and processed by mapper functions.

Each mapper reads its portion of the data and converts it into intermediate **key-value pairs**.

Shuffle and Sort Phase:

The system groups and sorts the intermediate key-value pairs based on their keys before sending them to the reducers.

Reduce

Phase:

In this phase, reducer functions process the grouped key-value pairs to produce the final aggregated output.

Apart from HDFS and MapReduce, Hadoop also includes additional components that extend its capabilities:

- **YARN (Yet Another Resource Negotiator):** Manages cluster resources and schedules tasks efficiently.
- **Hadoop Common:** Provides libraries and utilities required by Hadoop modules.
- **Hive:** A data warehousing tool that allows SQL-like queries on Hadoop datasets.
- **Pig:** A high-level scripting language used for data processing.
- **HBase:** A NoSQL database built on top of HDFS for real-time data access.

Hadoop clusters generally consist of two types of nodes: **Master Nodes** and **Worker Nodes**. The master node manages the overall cluster operations, including resource allocation and metadata management. Worker nodes perform actual data storage and processing tasks.

Before running any MapReduce program, Hadoop must be properly installed and configured on the system. This includes installing Java, configuring Hadoop environment variables, setting up HDFS directories, and starting Hadoop services such as **NameNode**, **DataNode**.

A commonly used example to understand the MapReduce programming model is the **Word Count program**, which counts the number of occurrences of each word in a large text dataset. In this example, the mapper outputs word-frequency pairs, and the reducer aggregates the counts to produce the final result.

Hadoop is widely used in industries for **large-scale data analytics, log processing, recommendation systems, fraud detection, social media analysis, and machine learning applications**. By combining distributed storage (HDFS) and parallel processing (MapReduce), Hadoop provides a powerful platform for managing and analyzing massive datasets efficiently.



Algorithm / Steps:

1. Install and configure Apache Hadoop on the local system.
2. Set Java and Hadoop environment variables and configure core-site.xml and hdfs-site.xml.
3. Format the NameNode using hdfs namenode -format.
4. Install and configure Apache Hadoop on the local system.
5. Set Java and Hadoop environment variables and configure core-site.xml and hdfs-site.xml.
6. Format the NameNode using hdfs namenode -format.
7. Start HDFS and YARN services.
8. Create a sample input text file for processing.
9. Write a simple MapReduce program (e.g., Word Count).
10. Compile the program and create a JAR file.
11. Run the MapReduce job using the Hadoop command.
12. View the output from HDFS using hdfs dfs -cat /output/part-r-00000.

Source Code:

// Mapper (Map Phase)

```
public void map(Object key, Text value, Context context) {  
    String[] words = value.toString().split(" ");  
    for (String word : words) {  
        context.write(new Text(word), new IntWritable(1));  
    }  
}
```

// Reducer (Reduce Phase)

```
public void reduce(Text key, Iterable<IntWritable> values, Context context) {  
    int sum = 0;  
    for (IntWritable val : values) {  
        sum += val.get();  
    }  
    context.write(key, new IntWritable(sum));  
}
```

Conclusion:

This experiment demonstrates how Apache Hadoop enables efficient storage and processing of large datasets in a distributed environment. By configuring Hadoop and executing a simple MapReduce program, we understood how data is divided, processed in parallel, and aggregated to



generate meaningful results. Implementing the Word Count example provided practical insight into the Map and Reduce phases and highlighted the importance of distributed computing in handling big data applications. The exercise also reinforced key Hadoop concepts such as HDFS block distribution, data locality, and fault tolerance, showing how the framework automatically manages node failures and ensures reliable processing. Overall, this foundational understanding of Hadoop's architecture and MapReduce programming model serves as a stepping stone for exploring more advanced big data technologies and real-world large-scale data analytics.

The hands-on implementation also emphasized the importance of proper configuration and environment setup in distributed systems. By observing how data blocks are replicated across nodes and processed in parallel, we gained valuable insight into the scalability and reliability that Hadoop provides for modern data-intensive applications.

Questions:

1. What is Apache Hadoop and why is it used in big data applications?
2. What are the main differences between HDFS and traditional file systems?
3. Explain the working of the MapReduce framework. How do the Map and Reduce phases function?
4. Why is it necessary to configure environment variables and format the NameNode before starting Hadoop services?
5. What is the purpose of executing a simple program like Word Count in Hadoop, and what insights does it provide about distributed data processing?